`llama-modchip`: Hardware-Aware Mathematical and Kernel Co-Design for 30B–284B Sparse MoE Inference at 256K Context on Commodity CPUs and GPU+CPU Hybrids

Paolo Colombo
Independent Systems & Machine Learning Researcher
`https://github.com/synchopate/llama-modchip`

November 2026

Abstract

Running frontier Sparse Mixture-of-Experts (MoE) language models at a 262,144-token (256K) context window is widely assumed to require multi-GPU clusters with high-bandwidth HBM memory. On commodity CPUs (50–90 GB/s dual-channel DDR5), standard inference engines fail due to four compounding bottlenecks: (1) quantizing 256-expert SwiGLU layers below 2 bits/weight without calibration data collapses reasoning, while standard 2-bit formats exceed 64 GiB RAM on 284B-parameter models; (2) neural draft models (0.5B–1B) for speculative decoding read 2–3 GB of weights per draft cycle, saturating the DDR5 bus and flushing the CPU's victim L3 cache; (3) four-level page-table walks across 4–14 million 4 KB pages thrash the L2 STLB; and (4) naive linear rescaling during dynamic MoE expert pruning distorts the variance ratio against dense shared experts.

We present `llama-modchip` (`v2.4.0`), an open-source hardware-aware inference engine that co-designs quantization formats, SIMD kernels, cache/TLB hierarchy management, draftless chess-engine speculative search, and norm-preserving low-rank/MoE mathematics. First, we introduce **Block-64 2-Level Hierarchical Non-Linear Quantization (`Fractal-64`, 3.75 bpw)** with single-cycle AVX-512 `vprorvq` and AVX2 `vpsllvd+vpmaddwd` sub-scale fusion, paired with **Analytical Second-Order `SwiGLU-H2` Hessian Compression**, shrinking the 284.33B-parameter `DeepSeek-V4-Flash` (13.17B active) from 76.87 GiB to **56.56 GiB** on disk and **58.9 GiB total `VmRSS` at full 256K context** on a single 64 GB CPU. Second, we replace neural draft models with **`RouteCrack + Stockfish-PVS v2`**, a 1.64 MB L2/L3-resident speculative engine combining an 8-level FNV-64 hash cascade with 4-way Zobrist transposition tables and Principal Variation Search, achieving **96.39% live draft acceptance** (48.39 tok/s average / 60.40 tok/s peak on a 16-core Zen 5 CPU, and **111.62 tok/s** on a 64-core Threadripper PRO 7985WX) on `Nemotron-3.5-30B-A3B`. Third, we prove that because routed MoE expert vectors in $\mathbb{R}^{4096}$ are nearly mutually orthogonal, tail expert pruning must use **L_2-quadrature rescaling** rather than linear L_1 rescaling, cutting next-token KL-divergence by **58.1%** (to 0.0042 nats / +0.42% PPL at Top-5) and pushing pure single-token CPU decode on `DeepSeek-V4-Flash 284B` from 5.91 tok/s to **9.06 tok/s (lossless)**, **9.68 tok/s (Top-5 L_2)**, and **10.12 tok/s (Top-4 L_2)**.

1 Introduction

Modern frontier language models increasingly adopt sparse Mixture-of-Experts (MoE) architectures paired with state-space or latent-attention compression to decouple total parameter capacity from per-token active compute. Two prominent examples illustrate this design space:

1. `NVIDIA-Nemotron-3.5-Lightning-30B-A3B` [1]: A 52-layer hybrid architecture (30.2B total, 3.0B active parameters) comprising 23 Mamba-2 State-Space Model (SSM) layers, 23 Sparse MoE layers (128 routed experts with top-6 routing plus 1 shared expert), and only 6 Grouped-Query Attention (GQA) layers (2 KV heads per layer, 12 KV heads total).

2. `DeepSeek-V4-Flash-0731` [2]: A 43-layer frontier MoE Transformer (284.33B total, 13.17B active parameters) combining 64-head Multi-Head Latent Attention (MLA, compressed KV latent rank 512, RoPE dimension 64, query LoRA rank 1024, and 8 output head-groups of rank 1024), sparse indexer compressors (`CSA/LID/HCA`), 4-stream Sinkhorn-20 Hyper-Connections (hc = 4), and 256 routed experts (top-6 active plus 1 shared expert per layer), where layers 0..2 employ deterministic token-ID hash routing (`tid2eid`) and layers 3..42 employ learned $\sqrt{\text{softplus}}$ top-k routing.

In principle, activating only 3.0B to 13.17B parameters per token makes these architectures attractive for local deployment on single-socket workstations without data-center GPUs. Prior systems for consumer hardware, such as PowerInfer [6] (which exploits ReLU activation sparsity via offline neuron profiling), KTransformers [7] (which offloads MoE experts to CPU while running attention on a GPU), SqueezeLLM [8] (sensitivity-based non-uniform quantization requiring calibration gradients), and FlashAttention-2 [9] (GPU-centric IO-aware attention tiling), have advanced local inference significantly. However, deploying modern SwiGLU/Mamba-2/MLA MoE models at a full 262,144-token (256K) context window on a pure 64 GB RAM CPU box (or a consumer 8–16 GB GPU + 16–64 GB RAM hybrid) without offline calibration datasets exposes four compounding bottlenecks that neither baseline `llama.cpp` [3] nor prior hybrid engines address:

- **The 64 GiB Capacity & Non-Divisibility Wall**: Even at aggressive 2-bit quantization (`IQ2_XXS`), `DeepSeek-V4-Flash` requires 76.87 GiB on disk, triggering immediate Linux OOM termination on a 64 GB host. Meanwhile, on `Nemotron-3.5-30B-A3B`, expert dimensions (1856×2688) are not divisible by standard K-quant superblock sizes ($QK_K = 256$), forcing `llama-quantize` to fall back to 4.5 bpw `Q4_0` (17.59 GiB).

- **The CPU Speculative Bandwidth Paradox**: On HBM3 GPUs (3.35 TB/s), a 1B neural draft model executes 5 steps in < 1 ms. On dual-channel DDR5 CPUs ($\sim$ 72 GB/s), streaming 600 MB of 1B draft weights 5 times reads 3.0 GB from DRAM ($\sim$ 40 ms)—slower than executing the 30B MoE target model itself (1.8 GB active), while evicting hot recurrent state and KV blocks from the 64 MB L3 victim cache.

- **Microarchitectural TLB & Graph-Rebuild Stalls**: Mapping 18–57 GiB of weights in default 4 KB pages creates 4.5–14.8 million page table entries (PTEs), exceeding the 3,072-entry L2 STLB of AMD Zen 5 by $1,500\times$–$4,800\times$. Furthermore, dynamic compressor topology changes in `DeepSeek-V4-Flash` trigger full 4,551-node compute-graph rebuilds on 7 out of every 8 decode tokens.

- **Unexamined Linear & Orthogonal Redundancy**: Nearly 31% of per-token DRAM traffic in `DeepSeek-V4-Flash` (991.6 MB/tok) is spent in the unactivated linear chain $\mathbf{W}_{o,b}\mathbf{W}_{o,a}$, while naive linear weight renormalization during dynamic MoE tail pruning distorts the amplitude ratio between routed experts and the dense shared expert.

1.1 Contributions

This paper makes four tightly coupled contributions, all implemented in open-source C/C++ and verified across x86-64 (AMD Zen 5, 8-CCD Zen 4 Threadripper, Intel AVX2), ARM64 (AWS Graviton3, Apple M4 Pro), and GPU+CPU hybrids (Linux/Windows CUDA & Vulkan):

1. **`Fractal-64` Quantization, `SwiGLU-H2` Analytical Hessian Compression, & AVX-512/AVX2 Kernel Co-Design (§2)**: We design a 30-byte Block-64 hierarchical non-linear format (3.75 bpw) whose 4-bit sub-scales fuse at zero instruction overhead into AVX2 `vpmaddwd` and AVX-512 `vprorvq` (`_mm512_rorv_epi64`), and derive a closed-form second-order SwiGLU Hessian metric that compresses `DeepSeek-V4-Flash 284B` to 56.56 GiB (58.9 GiB `VmRSS` at 256K context).

2. **Mathematical Co-Design for `DeepSeek-V4-Flash`: SVD Spectrum of $\mathbf{W}_{o,b}\mathbf{W}_{o,a}$ & L_2-Quadrature MoE Tail Pruning (§3)**: We prove that because routed expert outputs in $\mathbb{R}^{4096}$ are nearly orthogonal, pruned router weights must be rescaled in quadrature (α_{L_2}) rather than linearly (α_{L_1}), reducing next-token KL-divergence by 58.1% (to 0.0042 nats at Top-5) while preserving hash-routed lexical layers ($il = 0..2$). We also characterize the exact singular value spectrum of $\mathbf{W}_{o,b}\mathbf{W}_{o,a}$ (99.82% cosine similarity at rank $r = 768/1024$).

3. **Draftless Chess-Engine Speculative Decoding (`RouteCrack + Stockfish-PVS v2`, §4)**: We formulate the CPU-MoE speculative wall-clock equation and introduce a 1.64 MB L2/L3-resident drafter using 4-way Zobrist transposition tables and Principal Variation Search, unlocking Singular Horizon Extensions up to $M = 7$ with 98.74% marginal precision at $D = 5..7$.

4. **Cross-Platform Memory Hierarchy & Hybrid GPU+CPU Orchestration (§5)**: We combine 2 MB Transparent HugePages, bidirectional `_MM_HINT_NTA` victim-L3 isolation, 3D L1D cache tiling, zero-rebuild dynamic `CSA/LID` node skipping, 8-slot `Q8` activation caching, deterministic 4C/CCD multi-chiplet pinning, and live-VRAM Windows/Linux tensor-role splitting (`-n-cpu-moe`).

2 Quantization & SIMD Co-Design

2.1 Block-64 `Fractal-64` (`Q2_0`, 3.75 bpw)

In `Nemotron-3.5-30B-A3B`, the MoE projections have inner dimensions $1856 = 29 \times 64$ and $2688 = 42 \times 64$.

Because neither is divisible by $QK_K = 256$, standard K-quants (`Q3_K`, `IQ3_S`) cannot quantize the expert tensors. We introduce `Fractal-64` (mapped to the 64-weight `GGML_TYPE_Q2_0` slot), which encodes 64 weights in exactly **30 bytes (3.75 bits/weight)**:

- `ggml_half d` (2 bytes): FP16 super-block scale.
- `uint8_t scales[4]` (4 bytes): Eight 4-bit unsigned sub-scales $m_s \in \{0..15\}$ (representing shifted multipliers $\tilde{m}_s = m_s + 4 \in \{4..19\}$) covering eight 8-weight micro-blocks.
- `uint8_t qs[16]` (16 bytes) and `uint8_t qh[8]` (8 bytes): Lower 2 bits and upper 1 bit of 64 3-bit indices $q_i \in \{0..7\}$, mapped onto a symmetric non-linear codebook $\mathcal{C} = \{-14, -8, -4, -1, +1, +4, +8, +14\}$ fitted to the empirical Kurtosis of SwiGLU expert weights.

Zero-Overhead SIMD Sub-Scale Fusion. A naive implementation would convert each 8-weight dot product to FP32 before multiplying by $\tilde{m}_s$. Instead, against `Q8_0` activations $x_i \in [-127, 127]$, the maximum magnitude of an 8-element inner product is $8 \times 14 \times 127 = 14,224$. However, in our AVX2/AVX-512 kernel (`ggml_vec_dot_q2_0_q8_0_generic`), we accumulate pairs of products via `_mm256_maddubs_epi16` (`vpmaddubsw`), whose 16-bit signed lanes have a worst-case bound of $2 \times 14 \times 127 = 3,556$. Multiplying by the maximum sub-scale $\tilde{m}_s \leq 19$ yields $3,556 \times 19 = 67,564$, whereas `vpmaddwd` (`_mm256_madd_epi16(p01, vmult01)`) widens directly to 32-bit signed integers (`INT32_MAX` $= 2.14 \times 10^9$) while summing adjacent pairs! Thus, replicating $\tilde{m}_s$ into 16-bit lanes and feeding it as the second operand of `vpmaddwd` fuses sub-block rescaling and horizontal pairwise reduction in a **single clock cycle with zero extra instructions**. On Zen 5, extracting the 64-bit upper bit-plane `qh[8]` is further accelerated via single-instruction AVX-512 variable 64-bit rotation `_mm512_rorv_epi64` (`vprorvq`), reducing 10M-block contraction time from 86.5 ms to **55.8 ms (+55% speedup)**.

2.2 Analytical Second-Order `SwiGLU-H2` Compression for 284B MoE

To fit `DeepSeek-V4-Flash` (284.33B parameters) inside < 60 GiB of RAM without a multi-terabyte calibration run, we observe that each routed expert computes a gated bilinear product:

$$\mathbf{h}^{(e)}(\mathbf{x}) = \mathrm{SiLU}\Big(\mathbf{W}^{(e)}_{\text{gate}}\mathbf{x}\Big) \odot \Big(\mathbf{W}^{(e)}_{\text{up}}\mathbf{x}\Big). \tag{1}$$

Assuming isotropic unit-variance pre-activations $\mathbb{E}[\mathbf{x}\mathbf{x}^\top] \propto \mathbf{I}$, a second-order Taylor expansion of the reconstruction error $\|\mathbf{h}^{(e)}(\mathbf{x}) - \hat{\mathbf{h}}^{(e)}(\mathbf{x})\|_2^2$ with respect to quantization perturbations $\Delta\mathbf{W}^{(e)}_{\text{gate}}$ and $\Delta\mathbf{W}^{(e)}_{\text{up}}$ yields a closed-form expected diagonal Hessian importance weight for expert e, output row j:

$$I_{e,j} = \Big\|\mathbf{W}^{(e)}_{\text{gate}}[j,:]\Big\|_2^2 \cdot \Big\|\mathbf{W}^{(e)}_{\text{up}}[j,:]\Big\|_2^2. \tag{2}$$

In `scripts/transcode_dsv4.c`, we dequantize the paired Gate and Up expert slices simultaneously, weight `quantize_row_iq1_s` by $I_{e,j}$ on bulk routed layers, retain high-sensitivity anchor layers ($\{0, 1, 26, 28, 41, 42\}$ in `IQ2_XXS`; $\{2, 4, 5, 8, 27, 30, 32, 40\}$ in `IQ1_M`), and transcode all dense `Q8_0` MLA and Indexer projections (`attn_q_b`, `attn_output_a`, `attn_output_b`, `attn_compressor_*`, `indexer_compressor_*`) to `Fractal-64` (`Q2_0`, 3.75 bpw). This shrinks the 3-shard GGUF from 76.87 GiB down to **56.56 GiB** (-26.4%) and enables full 256K-context execution in **58.9 GiB** `VmRSS` on a 64 GB CPU host.

3 Mathematical Structure & Low-Rank / MoE Co-Design

3.1 Spectral Decay of the Unactivated Linear Chain $\mathbf{W}_{o,b}\mathbf{W}_{o,a}$

During single-token decode of `DeepSeek-V4-Flash`, $\sim$ 3.20 GB of weights are streamed from DDR5 per token. Profiling reveals that **31.0% of all DRAM traffic (991.6 MB/tok, or 23.06 MB/layer across 43 layers)** occurs in the MLA output projection (`deepseek4.cpp`), which applies two consecutive `Q2_0` matrices across $G = 8$ head-groups ($r_o = 1024$ inner dimension per group, $d = 4096$ model dimension) with **zero non-linearity, zero normalization, and zero bias** between them:

$$\mathbf{y}_{\text{attn}} = \sum_{g=1}^{8} \mathbf{W}^{(g)}_{o,b}\Big(\mathbf{W}^{(g)}_{o,a}\mathbf{x}_g\Big), \tag{3}$$

where $\mathbf{W}^{(g)}_{o,a} \in \mathbb{R}^{1024\times4096}$ and $\mathbf{W}^{(g)}_{o,b} \in \mathbb{R}^{4096\times1024}$.

De-quantizing the actual GGUF tensors (`blk.{0,3,4,10}.attn_output_{a,b}.weight`) and computing the exact Singular Value Decomposition $\mathbf{M}_g = \mathbf{W}^{(g)}_{o,b}\mathbf{W}^{(g)}_{o,a} = \mathbf{U}_g\mathbf{\Sigma}_g\mathbf{V}_g^\top$ across all 8 head-groups demonstrates rapid spectral decay ($\sigma_0 = 15.398 \rightarrow \sigma_{768} = 0.901 \rightarrow \sigma_{1023} = 0.048$; in early layers, the bottom 180 singular values fall below 10^{-3}). As shown in Table 2, rank-768 truncation (-25.0% DRAM $= -247.9$ MB/tok, saving ~ 6.9 ms/tok) retains **99.64% of Frobenius energy** and achieves **0.998204 output cosine similarity**.

3.2 Orthogonal Geometry of MoE Tail Pruning & L_2-Quadrature Rescaling

In each layer $l \in \{0..42\}$ of `DeepSeek-V4-Flash`, the FFN block sums one dense shared expert $\mathbf{e}_{\text{shared}}(\mathbf{x})$

Table 1: Exact operator-by-operator single-token decode profile (`LLAMA_PROFILE_OPS`, $N = 1$, 4,551 graph nodes) and 512-token prompt prefill (`pp512`) on `DeepSeek-V4-Flash-0731-Fractal` (284.33B total / 13.17B active, 3 transcoded GGUF shards = 56.56 GiB from the 4-shard 76.87 GiB `IQ2_XXS` source) on a single 16C/32T AMD Ryzen 9 9955HX (Zen 5, 0 GPUs).

Operator / Subsystem (43 Layers)	Types	Calls/Tok	`v2.3.0` Init	Pre-Micro-Opt	`v2.4.0` Lossless	`v2.4.0` Top-4 L2
1. MLA & Shared Expert `MUL_MAT` (`wq_b`, `wo_a/b`, `shexp`)	`q2_0`	512	78.40 ms	62.20 ms	**61.81 ms**	**61.81 ms**
2. Routed MoE `MUL_MAT_ID` (Bulk `SwiGLU-H2`)	`iq1_s`	87	25.10 ms	22.79 ms	**22.27 ms**	**15.17 ms** (−33.4%)
3. Routed MoE `MUL_MAT_ID` (Anchor Layers)	`iq2_xxs`	18	4.50 ms	4.04 ms	**4.01 ms**	**2.75 ms** (−31.9%)
4. Routed MoE `MUL_MAT_ID` (Mid-Sensitivity Layers)	`iq1_m`	24	2.10 ms	1.81 ms	**1.80 ms**	**1.16 ms** (−35.9%)
5. Output Head (`lm_head`) + Router Gates	`q4_K/q6_K/f32`	298	11.20 ms	9.29 ms	**9.05 ms**	**9.05 ms**
6. `HADAMARD` (FWHT), `RMS_NORM`, `HC_PRE/POST`, `FA`	`f32/f16`	1,810	19.60 ms	10.12 ms	**9.85 ms**	**8.90 ms**
7. Graph Rebuild & Dynamic `CSA/LID` Compressor Skip	Graph/C++	1	28.40 ms	11.67 ms	**1.54 ms**	**0.00 ms**
Total Single-Token Decode Latency ($N = 1$)	**All**	**4,551**	**169.30 ms**	**121.92 ms**	**110.33 ms**	**98.84 ms**
Pure Single-Token Decode Throughput (`tok/s`)	—	—	5.91 tok/s	8.20 tok/s	**9.06 tok/s** (+53.3%)	**10.12 tok/s** (+71.2%)
Batched Prompt Prefill Throughput (`pp512`)	—	5,375	24.70 tok/s	40.91 tok/s	**41.58 tok/s** (+68.3%)	**41.58 tok/s** (+68.3%)

Table 2: Exact SVD truncation analysis of the linear operator $\mathbf{M}_g = \mathbf{W}_{o,b}^{(g)}\mathbf{W}_{o,a}^{(g)}$ across 8 head-groups in `DeepSeek-V4-Flash`.

Rank r/1024	DRAM Saved	Frobenius Energy	Cosine Sim	Rel. L_2 Err
$r = 1024$ (Full)	0 MB/tok	100.00%	1.000000	0.00%
$r = 768$ (−25.0%)	−247.9 MB/tok	**99.64%**	**0.998204**	5.99%
$r = 640$ (−37.5%)	−371.9 MB/tok	98.61%	**0.992999**	11.80%
$r = 512$ (−50.0%)	−495.8 MB/tok	96.02%	0.979718	20.02%

and $K = 6$ routed experts $\mathbf{e}_{k_1}(\mathbf{x}), \ldots, \mathbf{e}_{k_6}(\mathbf{x}) \in \mathbb{R}^d$ ($d = 4096$):

$$\mathbf{y}_{\text{FFN}}(\mathbf{x}) = \mathbf{e}_{\text{shared}}(\mathbf{x}) + \sum_{i=1}^{K} w_i\, \mathbf{e}_{k_i}(\mathbf{x}), \quad \sum_{i=1}^{K} w_i = 1.5, \tag{4}$$

where $w_1 \geq w_2 \geq \cdots \geq w_K > 0$ are the sorted router weights. Logging $w_{1..6}$ across all 43 layers reveals a fundamental structural dichotomy:

1. **Hash-Routed Lexical Layers** ($l \in \{0, 1, 2\}$): Selected via 6 deterministic token-ID hash tables (`ffn_gate_tid2eid`). Their weights are nearly flat (e.g., $l = 2$: $[0.270, 0.261, 0.258, 0.257, 0.255, 0.199]$). Pruning any expert in $l \in \{0, 1, 2\}$ discards 1/6th of the token's lexical representation and corrupts numeric/hexadecimal constants. Thus, $l \in \{0, 1, 2\}$ must remain unpruned ($k = 6$).

2. **Learned-Router Layers** ($l \in \{3..42\}$): Selected via $\sqrt{\text{softplus}}$ top-k routing. The weights obey a steep power law (e.g., $l = 19$: $[0.762, 0.202, 0.156, 0.145, 0.119, 0.117]$; $l = 8$: $[0.446, 0.386, 0.367, 0.120, 0.092, 0.088]$). Across $l \in \{3..42\}$, the top-4 experts hold 81.2% of $\sum w_i$, the top-5 hold 91.0%, and the 6th expert carries only 6%–9% ($< 4\%$ of total FFN energy including $\mathbf{e}_{\text{shared}}$).

Proposition 1 (Suboptimality of Linear L_1 Rescaling vs. L_2-Quadrature Rescaling)**.** *Let $\mathbf{e}_{k_1}(\mathbf{x}), \ldots, \mathbf{e}_{k_K}(\mathbf{x}) \in \mathbb{R}^d$ ($d = 4096$) be zero-mean expert outputs with approximately equal second moments $\mathbb{E}\|\mathbf{e}_{k_i}\|_2^2 = \sigma_e^2$ and near-mutual orthogonality $\mathbb{E}\langle \mathbf{e}_{k_i}, \mathbf{e}_{k_j} \rangle \approx 0$ for $i \neq j$, and let $\mathbf{e}_{shared}(\mathbf{x})$ be orthogonal to the routed experts with $\mathbb{E}\|\mathbf{e}_{shared}\|_2^2 = \sigma_s^2$. When truncating the routed sum to $1 \leq k \leq K$ experts with a rescaling factor $\alpha > 0$, $\hat{\mathbf{y}}_{FFN}(\alpha) = \mathbf{e}_{shared} + \alpha \sum_{i=1}^{k} w_i \mathbf{e}_{k_i}$:*

1. *The minimum mean-squared error (L_2-optimal) projection onto span$(\mathbf{e}_{k_1}, \ldots, \mathbf{e}_{k_k})$ is achieved at $\alpha_{proj} = 1$, whereas preserving the exact expected squared norm $\mathbb{E}\|\hat{\mathbf{y}}_{FFN}(\alpha)\|_2^2 = \mathbb{E}\|\mathbf{y}_{FFN}\|_2^2 = \sigma_s^2 + \sigma_e^2 \sum_{i=1}^{K} w_i^2$ entering the downstream `RMSNorm` requires L_2-**quadrature rescaling**:*

$$\alpha_{L_2} = \sqrt{\frac{\sum_{i=1}^{K} w_i^2}{\sum_{i=1}^{k} w_i^2}}. \tag{5}$$

2. *Conversely, conventional linear L_1-sum rescaling $\alpha_{L_1} = \frac{\sum_{i=1}^{K} w_i}{\sum_{i=1}^{k} w_i}$ strictly inflates the variance of the routed MoE branch relative to the unscaled shared expert $\mathbf{e}_{shared}$ for all $1 \leq k < K$ by the factor:*

$$\Gamma(w) = \frac{\alpha_{L_1}^2}{\alpha_{L_2}^2} = \frac{\left(\sum_{i=1}^{K} w_i\right)^2}{\left(\sum_{i=1}^{k} w_i\right)^2} \cdot \frac{\sum_{i=1}^{k} w_i^2}{\sum_{i=1}^{K} w_i^2} > 1. \tag{6}$$

Proof. By mutual orthogonality, $\mathbb{E}\|\mathbf{y}_{\text{FFN}} - \hat{\mathbf{y}}_{\text{FFN}}(\alpha)\|_2^2 = \sigma_e^2 \left[(1-\alpha)^2 \sum_{i=1}^{k} w_i^2 + \sum_{i=k+1}^{K} w_i^2\right]$, which is minimized at $\alpha = 1$, while setting $\mathbb{E}\|\sum_{i=1}^{k}(\alpha w_i)\mathbf{e}_{k_i}\|_2^2 = \sigma_e^2 \alpha^2 \sum_{i=1}^{k} w_i^2 = \sigma_s^2 \sum_{i=1}^{K} w_i^2$ yields $\alpha = \alpha_{L_2}$. For typical learned-router weights at $k = 4$ ($\sum_{i=1}^{6} w_i = 1.50$, $\sum_{i=1}^{4} w_i = 1.22$, $\sum_{i=1}^{6} w_i^2 = 0.4806$, $\sum_{i=1}^{4} w_i^2 = 0.4414$), linear rescaling gives $\alpha_{L_1} = 1.2295$ (+23.0% amplitude inflation, $\Gamma = 1.388$), whereas quadrature rescaling gives $\alpha_{L_2} = \sqrt{0.4806/0.4414} = 1.0435$ (+4.35%). □

Remark 1 (Positive Directional Correlation ρ and Downstream `RMSNorm` SNR)**.** *If co-selected experts share a small positive pairwise directional correlation $\rho = \mathbb{E}\langle \mathbf{e}_{k_i}, \mathbf{e}_{k_j} \rangle / \sigma_e^2 \in [0, 1)$ (or a common non-zero mean $\boldsymbol{\mu}$),*

the exact variance-preserving factor generalizes to

$$\alpha_{gen}(\rho) = \sqrt{\frac{(1-\rho)\sum_{i=1}^{K} w_i^2 + \rho\left(\sum_{i=1}^{K} w_i\right)^2}{(1-\rho)\sum_{i=1}^{k} w_i^2 + \rho\left(\sum_{i=1}^{k} w_i\right)^2}},$$

which smoothly interpolates between α_{L_2} ($\rho \to 0$) and α_{L_1} (the degenerate collinear limit $\rho \to 1$). Moreover, although $\alpha_{proj} = 1$ minimizes pre-normalization Euclidean MSE, the FFN output is immediately normalized by RMSNorm($\mathbf{r} + \mathbf{y}_{FFN}$). Leaving $\alpha = 1$ reduces the variance of the routed MoE branch relative to the residual stream $\mathbf{r}$ and shared expert $\mathbf{e}_{shared}$, whereas α_{L_2} preserves the exact signal-to-noise energy ratio between the routed and shared branches prior to `RMSNorm` division.

Table 3 validates Proposition 1 empirically on `DeepSeek-V4-Flash 284B` on a deterministic code-generation micro-benchmark (24 teacher-forced single-token decode steps over an algorithmic Python/C prompt plus 180-token free generation, where $D_{\mathrm{KL}}(P\|Q)$ and the local perplexity shift $\Delta\mathrm{PPL}_{\text{top-10}} = \exp(D_{\mathrm{KL}}) - 1$ are evaluated over the renormalized top-10 logit support): switching from α_{L_1} to α_{L_2} at $k = 4$ **cuts top-10 next-token KL-divergence by 58.1%** (0.0671 → 0.0281 nats) and restores 100.0% Top-3 token agreement, while $k = 5$ (α_{L_2}) achieves **9.68 tok/s** with a negligible **0.0042 nats top-10 KL-divergence (+0.42% local $\Delta\mathrm{PPL}_{\text{top-10}}$)** and **0.99714 probability cosine similarity**.

4 Draftless Speculative Decoding: `RouteCrack + Stockfish-PVS v2`

4.1 The CPU-MoE Speculative Golden Equation

In standard speculative decoding [4], D draft tokens are verified in a single batched target forward pass. On a GPU where $D \leq 8$ lies inside the compute-bound roof, verification time equals a single-token step ($c_{\text{gpu}} \approx 0$). On a memory-bandwidth-bound CPU hosting a Sparse MoE, each additional token in a verification batch activates additional routed experts from DDR5 ($\sim 0.30\times$ single-token latency per drafted token on `Nemotron-3.5-30B-A3B`), and any rejected draft token triggers an 18.4 MB Mamba-2 recurrent state rollback (`slot.spec_ckpt`). Consequently, net wall-clock gain over A accepted tokens and $W = D - A$ wasted tokens obeys:

$$\mathrm{WallGain}_{\mathrm{CPU}} = A - 0.30\,D = 0.70\,A - 0.30\,W. \quad (7)$$

A 1B neural draft model reading 600 MB per step incurs $\sim$ 40 ms of DDR5 latency for $D = 5$ and flushes the 64 MB L3 cache, yielding 11.5 tok/s (slower than non-speculative decoding).

4.2 `StockfishDrafter`: 192 KB L2 Zobrist TT & Principal Variation Search

To achieve high recall without DRAM traffic, `RouteCrack` (`rc_predictor.h`) pairs a static 8-level FNV-64 hash cascade, an online session trie (`rc_trie_v2.h`), and an 11,382-entry multi-token macro codebook with **`StockfishDrafter` (`rc_stockfish.h`)** [5]. `StockfishDrafter` maintains three lockless 4-way bucketed Zobrist transposition tables (`tt5`, `tt4`, `tt3` = 3 × 64 KB = 192 KB, < 19% of Core 0's 1 MB L2 cache on Zen 5) with 32-bit verification keys, verified `best_move`, explicit `refuted_move` tracking, and history gravity ($\in [-512, +512]$).

Crucially, because static cascade tables and live-session Zobrist transposition tables have *uncorrelated error modes*, gating uncertain candidates on their orthogonal agreement unlocks four previously unusable high-recall predictors: (1) Online Session Trie Rescue (92.19% marginal precision), (2) Top-4 Cascade Disambiguation, (3) Multi-Token Macro Codebook chains (96.44% precision), and (4) **Singular Horizon Extension up to $M = 7$ tokens/step** (98.74% marginal precision at $D = 5..7$). As shown in Table 4 (evaluated across 77,456 held-out tokens in `test_synergy.cpp`), `Stockfish-PVS v2` adds +223 **accepted tokens with zero net additional wasted tokens ($W = 892 \to 892$, 100.0% net marginal precision)** and eliminates 32 Mamba-2 rollbacks.

5 Microarchitectural & Multi-Platform Systems Co-Design

5.1 Kernel & Graph Optimizations in `ggml-cpu`

1. **Zero-Rebuild Dynamic `CSA/LID` Skip (`ggml_dsv4_skip_csa_comp`)**: In `DeepSeek-V4-Flash`, `CSA/LID` compressor blocks execute only once every 8 tokens during decode. Instead of rebuilding the 4,551-node compute graph when transitioning between compressor and non-compressor steps, we build the superset graph once, tag dummy compressor nodes with `flags |= 0x100`, and skip them in $O(1)$ inside `ggml_graph_compute_thread`, eliminating 26.8 ms/tok of graph allocation overhead.

2. **8-Slot Thread-Local `Q8_K/Q8_0` Activation Cache & `MUL_MAT_ID` Row-Group Reuse**: When multiple projections share the same FP32 input tensor (e.g., `wq_a` + `wkv_a`, or `up_exps` + `gate_exps` + `shexp`), `s_q8_cache_buf` quantizes the activation once and reuses it across subsequent contractions, while `g_tls_mmid_ids_valid` reuses the

Table 3: Empirical quality vs. decode speed of runtime MoE tail pruning on `DeepSeek-V4-Flash 284B` ($l \in \{0..2\}$ kept at 6/6 experts; evaluated over 24 teacher-forced code-generation decode steps + 180-token free generation on 16C Zen 5, with D_{KL} and $\Delta\mathrm{PPL}_{\text{top-10}} = \exp(D_{\mathrm{KL}}) - 1$ measured on the renormalized top-10 logit support).

Runtime MoE Configuration	Decode (tok/s)	Latency	Speedup vs. Init	Prob CosSim	Top-10 D_{KL} (nats)	Local $\Delta\mathrm{PPL}_{\text{top-10}}$	Top-1 / Top-3 Match
0. `v2.3.0` Initial Baseline ($N = 1$)	5.91 tok/s	169.30 ms	1.00×	1.00000	0.0000	+0.00%	100.0%/100.0%
1. **`v2.4.0` Lossless (`Top-6` All, Default)**	**9.06 tok/s**	**110.33 ms**	**+53.3%**	**1.00000**	**0.0000**	**+0.00%**	**100.0%/100.0%**
2. **Adaptive Tail** ($w_i < 0.11$, α_{L_2})	**9.52 tok/s**	**105.02 ms**	**+61.1%**	**0.99634**	**0.0047**	**+0.47%**	**95.8%/100.0%**
3. **Sweet-Spot (`Top-5` on** $l \geq 3$, α_{L_2})	**9.68 tok/s**	**103.27 ms**	**+63.8%**	**0.99714**	**0.0042**	**+0.42%**	**95.8%/100.0%**
4. **Hybrid `6/4/6`** ($l \in [16, 39]$ **at 4,** α_{L_2})	**9.83 tok/s**	**101.74 ms**	**+66.3%**	**0.99660**	**0.0184**	**+1.86%**	**95.8%/100.0%**
5. **Hybrid `5/4/5`** ($l \in [12, 40]$ **at 4,** α_{L_2})	**9.95 tok/s**	**100.55 ms**	**+68.4%**	**0.99463**	**0.0215**	**+2.17%**	**95.8%/100.0%**
6. **Turbo `10+` (`Top-4` on** $l \geq 3$, α_{L_2})	**10.12 tok/s**	**98.84 ms**	**+71.2%**	**0.99587**	**0.0281**	**+2.85%**	**95.8%/100.0%**
Ablation: `Top-4` with Linear L_1 Rescaling (α_{L_1})	10.10 tok/s	98.97 ms	+70.9%	0.98698	0.0671	+6.94%	95.8%/95.8%

Table 4: Held-out ablation across 77,456 `Nemotron-3.5` tokens (351 contexts, `speculative/cpp/test_synergy.cpp`).

Drafter Configuration	Draft D	Acc A	Waste W	Rollbacks	WallGain
1. `RouteCrack` ($M = 5$)	6,332	5,440	892	482	+3,540.4
2. `+ Stockfish-PVS v1`	6,329	5,459	**870**	461	+3,560.3
3. `+ Stockfish-PVS v2` ($M = 7$)	**6,555**	**5,663**	**892**	**450**	**+3,696.5**

bucketed expert row mapping between `up_exps` and `gate_exps`.

3. **In-Register 9-Stage AVX2 Fast Walsh-Hadamard Transform & Vectorized `HC_PRE/POST`**: `DeepSeek-V4-Flash` invokes a 512-point FWHT (`GGML_OP_HADAMARD`) 396 times per token. We keep all 9 butterfly stages ($s = 1, 2, \ldots, 256$) inside AVX2 `YMM` registers using lane permutes (`_mm256_permute2f128_ps`, `_mm256_shuffle_ps`) and vectorize the 4-stream Sinkhorn-20 Hyper-Connections (`HC_PRE/POST`), cutting non-matrix operator time in half (19.60 → 9.85 ms/tok).

4. **2 MB Transparent HugePages, 3D L1D Tiling, & Bidirectional `_MM_HINT_NTA` L3 Isolation**: Allocating weight maps and compute arenas with 2 MB-aligned `MADV_HUGEPAGE` expands Zen 5 L2 STLB coverage to 6.14 GB (eliminating 99.6% of page-table walks). Streaming cold MoE weights and KV tokens older than `LLAMA_KV_L3_WINDOW` (2,048 on Nemotron; 1,024 on DeepSeek MLA with 8-step sparse lookahead prefetching) via `_mm_prefetch(..., _MM_HINT_NTA)` prevents L3 victim-cache pollution.

5. **Cross-Platform Thread-Safe Runtime Control (`Linux/macOS mmap` & `Windows _WIN32`)**: Runtime MoE parameters ($k_{\text{early}}, k_{\text{mid}}, w_{\min}$, scale_mode) are initialized on thread 0 with release-acquire atomic synchronization (`atomic_int s_moe_topk`, `s_shared_moe_skip_mask`) and read from a 32-byte shared file mapping (`/tmp/dsv4_moe_cfg.bin` with `O_NOFOLLOW` and `DSV4_MOE_CFG_PATH` override via POSIX `mmap`, and `%TEMP%\dsv4_moe_cfg.bin` via Win32 `CreateFileMappingA`/`MapViewOfFile`), enabling sub-nanosecond (0.3 ns) L1-cached inspection with zero syscalls per token while resetting the skip mask across batched prefill passes ($N > 1$).

5.2 Multi-Chiplet Scaling & Hybrid GPU+CPU Tensor-Role Splitting

Table 5 summarizes cross-platform scaling:

- **8-CCD Threadripper PRO 7985WX (4C/CCD Law)**: Launching 64 threads (8C/CCD) across 8 CCDs saturates the Infinity Fabric with atomic work-stealing traffic, dropping decode speed from 80.14 tok/s to 51.80 tok/s. Selecting 4 physical cores per CCD (`-t 32`) via sysfs L3 topology discovery yields **386.00 tok/s prefill**, **80.14 tok/s cold decode**, and **111.62 tok/s warm decode**.

- **Hybrid GPU+CPU Tensor-Role Splitting (`-ngl 99 -n-cpu-moe` N `-load-mode none`)**: On 11–16 GB GPUs (Linux Vulkan or Windows CUDA), offloading by tensor role keeps 100% of the dense backbone, KV cache, and Mamba-2 `SSM_SCAN` in high-bandwidth VRAM while placing N sparse MoE layers in System RAM. On an AMD RX 9070 XT 16GB, `Fractal-MXFP4` (15.2 GiB) achieves **75.1 tok/s (new code) / 80.8 tok/s (in-context refactor)** at 256K context, outperforming `Q4_0` (69.7 / 76.2 tok/s at 16.1 GiB) while saving 0.9 GB of VRAM. On Windows 10/11 (`windows/modchip_win.ps1`), live free VRAM is queried via `nvidia-smi` with a 550 MB WDDM safety reserve to prevent PCIe GTT paging.

6 Limitations & Negative Results

1. **CPU-Only Scope of `Fractal-64` (`Q2_0`)**: Because `Fractal-64` repurposes the `Q2_0` type ID with a custom 30-byte Block-64 layout supported in `ggml-cpu`, offloading `Q2_0` MoE tensors to Vulkan/CUDA GPUs without a matching GPU shader produces invalid outputs; hybrid GPU+CPU setups should use `Fractal-MXFP4` or `Q4_0`, which have native Vulkan and CUDA shaders.

Table 5: Multi-platform verification across x86-64 (Zen 5, 8-CCD Zen 4 Threadripper), ARM64 (AWS Graviton3, Apple M4 Pro), and GPU+CPU Hybrids (Linux/Windows Vulkan & CUDA) on `Nemotron-3.5-30B-A3B` and `DeepSeek-V4-Flash 284B`.

Hardware Platform	Cores / Cache / GPU	Memory	Model & Profile	Prefill (tok/s)	Base / Cold Decode	Warm / Burst Decode
AMD Threadripper PRO 7985WX	64C/128T (8 CCDs, 256MB L3)	256GB 8-Ch DDR5	Nemotron-30B (`-t 32` 4C/CCD)	**386.00**	**73.84 – 80.14**	**111.62 tok/s**
AMD 7945HX3D + RX 9070 XT	16C (128MB L3) + 16GB Vulkan	94GB DDR5 + 16GB	Nemotron-30B (`MXFP4` 256K)	**563.00**	**75.10 tok/s**	**80.80 tok/s**
AMD Ryzen 9 9955HX (Zen 5)	16C/32T (2 CCDs, 64MB L3)	64GB DDR5 (0 GPU)	Nemotron-30B (`Fractal-64`)	**241.0 – 306.8**	**25.02 – 34.52**	**48.39 / 60.40 tok/s**
Apple M4 Pro (Unified SoC)	14C CPU + 20C Metal GPU	24GB Unified (0 Swap)	Nemotron-30B (128K Ctx)	**256.0 – 549.0**	**47.0 – 48.0**	**49.0+ tok/s**
AWS Graviton3 `m7g.2xlarge`	8 vCPU ARM64 Neoverse-V1	31GB DDR5 (0 GPU)	Nemotron-30B (`-t 8`)	**101.67**	**41.05 tok/s**	**44.97 tok/s**
Intel i9-9900K + RTX 2080 Ti	8C/16T + 11GB CUDA (Win 11)	16GB DDR4 + 11GB	Qwen3-14B / Nemotron-8B / 30B	Auto CUDA 12.4	**50.0 – 58.0**	**100+ tok/s** (8B)
AMD Ryzen 9 9955HX (Zen 5)	16C/32T (2 CCDs, 64MB L3)	64GB DDR5 (58.9GiB RSS)	**DeepSeek-V4 284B (`Fractal`)**	**41.58** (32.6ms Slot0)	**9.06 – 10.12**	**11.50 – 14.69 tok/s**

2. **Non-Temporal Prefetching on 50-Byte Unaligned `iq1_s` Blocks**: Issuing `_MM_HINT_NTA` inside `ggml_vec_dot_iq1_s_q8_K` degraded `iq1_s` throughput by $\sim$ 5% because `sizeof(block_iq1_s)` = 50 bytes ($<$ 64-byte cache line), causing premature eviction of cache lines straddling adjacent blocks.

3. **Catastrophic Cliff Below Top-4 Routing**: While L_2-quadrature pruning from Top-6 to Top-5 (+0.42% PPL) or Top-4 (+2.85% PPL) preserves generation quality, truncating to Top-2 alters the covariance spectrum across 43–52 layers and causes EOS suppression.

7 Conclusion

By co-designing hierarchical non-linear quantization (`Fractal-64` and analytical `SwiGLU-H2`), AVX-512/AVX2 kernels, L2/L3-resident chess-engine speculative search (`Stockfish-PVS v2`), and L_2-quadrature MoE / low-rank mathematics, `llama-modchip` enables frontier 30B–284B Sparse MoE inference at 256K context on commodity 64 GB CPUs and consumer GPU+CPU hybrids. Full source code, C/C++ patches, and reproducible benchmarks are available at https://github.com/synchopate/llama-modchip.

References

[1] NVIDIA, "Nemotron-H and Nemotron-3.5: Hybrid Mamba-Transformer Mixture-of-Experts Models for Long-Context Reasoning," *arXiv preprint*, 2025.

[2] DeepSeek-AI, "DeepSeek-V3 / V4 Technical Report: Multi-Head Latent Attention and Auxiliary-Loss-Free Load Balancing," *arXiv preprint arXiv:2412.19437*, 2024–2025.

[3] G. Gerganov et al., "`llama.cpp`: LLM Inference in C/C++," https://github.com/ggml-org/llama.cpp, 2023–2026.

[4] Y. Leviathan, M. Kalman, and Y. Matias, "Fast Inference from Transformers via Speculative Decoding," in *Proc. ICML*, 2023.

[5] T. Romstad, M. Costalba, J. Kiiski, et al., "Stockfish: Open-Source Chess Engine (Principal Variation Search and Zobrist Transposition Tables)," https://stockfishchess.org, 2008–2026.

[6] Y. Song et al., "PowerInfer: Fast Large Language Model Serving with a Consumer-Grade GPU," in *Proc. ACM SOSP*, 2024.

[7] H. Chen et al., "KTransformers: Unleashing the Full Potential of CPU/GPU Hybrid Inference for MoE Models," *arXiv preprint*, 2024–2025.

[8] S. Kim et al., "SqueezeLLM: Dense-and-Sparse Quantization," in *Proc. ICML*, 2024.

[9] T. Dao, "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning," in *Proc. ICLR*, 2024.